智能学习助手 · 技术路线图

基于通义千问 + 自研 ReAct + FAISS 的 RAG 智能体系统

一、五层架构

表现层 Presentation	Streamlit Web 界面 / 多页面路由 / 思维链可视化
应用层 Application	自研 ReAct Agent 循环 / 事件流式输出
能力层 Capability	Function Calling / 知识库检索 / 对话管理
模型层 Model	通义千问 qwen-max (DashScope OpenAI 兼容接口)
基础设施层 Infra	FAISS 向量库 / sentence-transformers / PyPDF2

二、完整问答数据流（ReAct 循环）

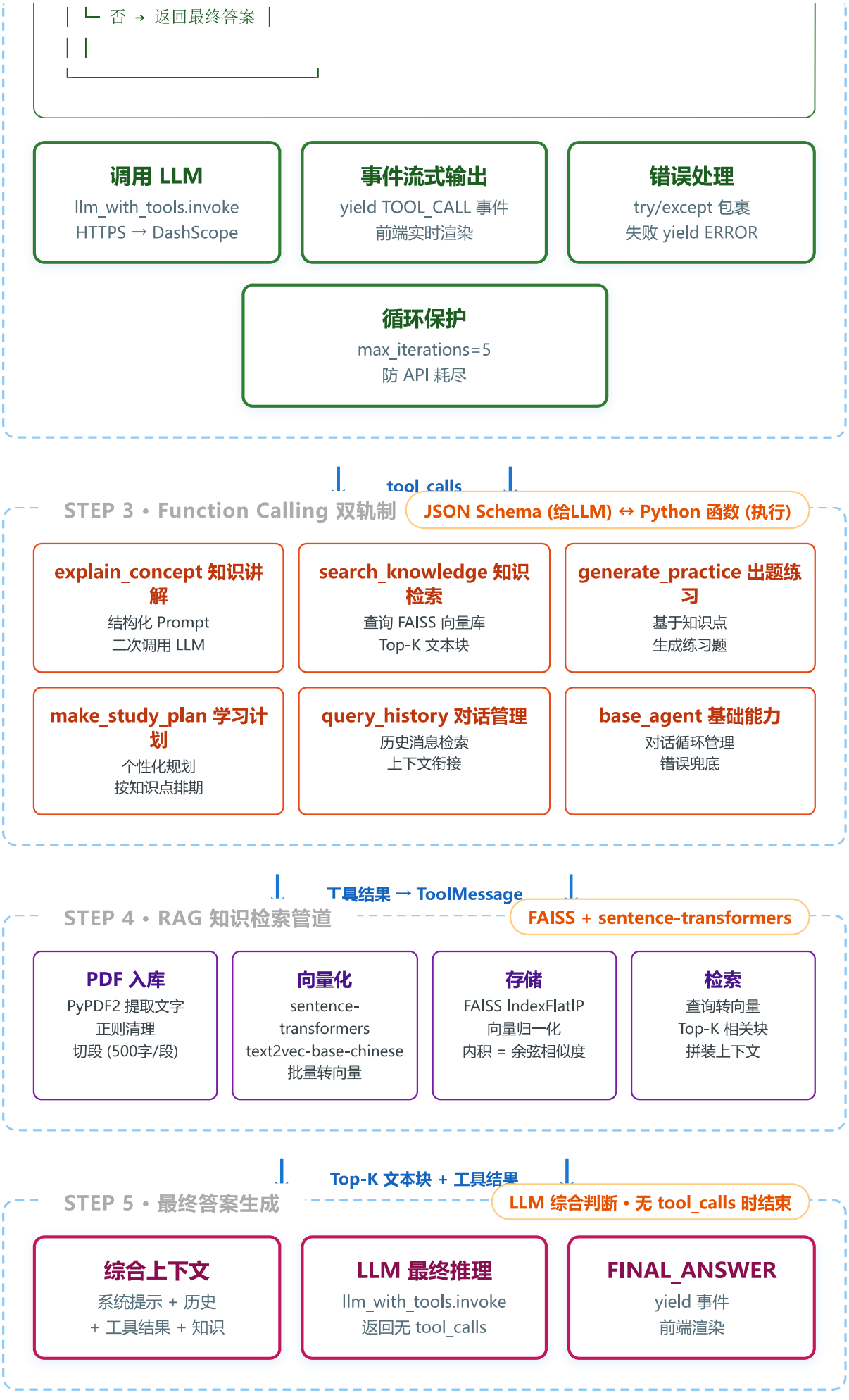
STEP 1 · 用户输入 — Streamlit 前端

用户提问	状态管理	消息构建
浏览器输入 "什么是递归"	st.session_state 缓存 Agent + 历史	SystemMessage + HumanMessage

react_stream()

STEP 2 · ReAct Agent 循环 (max_iterations=5) — 自研循环 · 非黑盒

```
[构建消息: 系统提示 + 历史 + 用户问题]
↓
┌──────── 循环开始 ────────┐
│                          │
│ 发送给 LLM (带工具定义)  │
│ ↓                        │
│ LLM 返回响应             │
│ ↓                        │
│ 有 tool_calls?           │
│ ├─ 是 → 执行工具         │
│ │ → 结果塞回消息         │
│ │ → 继续循环             │
```


└ 否 → 返回最终答案
调用 LLM
llm_with_tools.invoke
HTTPS → DashScope
事件流式输出
yield TOOL_CALL 事件
前端实时渲染
错误处理
try/except 包裹
失败 yield ERROR
循环保护
max_iterations=5
防 API 耗尽
tool_calls
STEP 3 • Function Calling 双轨制
JSON Schema (给LLM) ↔ Python 函数 (执行)
explain_concept 知识讲解
结构化 Prompt
二次调用 LLM
search_knowledge 知识检索
查询 FAISS 向量库
Top-K 文本块
generate_practice 出题练习
基于知识点
生成练习题
make_study_plan 学习计划
个性化规划
按知识点排期
query_history 对话管理
历史消息检索
上下文衔接
base_agent 基础能力
对话循环管理
错误兜底
工具结果 → ToolMessage
STEP 4 • RAG 知识检索管道
FAISS + sentence-transformers
PDF 入库
PyPDF2 提取文字
正则清理
切段 (500字/段)
向量化
sentence-transformers
text2vec-base-chinese
批量转向量
存储
FAISS IndexFlatIP
向量归一化
内积 = 余弦相似度
检索
查询转向量
Top-K 相关块
拼装上下文
Top-K 文本块 + 工具结果
STEP 5 • 最终答案生成
LLM 综合判断 • 无 tool_calls 时结束
综合上下文
系统提示 + 历史
+ 工具结果 + 知识
LLM 最终推理
llm_with_tools.invoke
返回无 tool_calls
FINAL_ANSWER
yield 事件
前端渲染

三、关键技术决策

5 个核心决策：

- **① 自研 ReAct 循环**（非 AgentExecutor）— 为事件流式输出 + 思维链可视化 + 细粒度错误控制，代价是代码量大
- **② FAISS 取代 pgvector/Chroma** — 免 PostgreSQL 安装、部署简单、本地运行快
- **③ OpenAI 兼容接口接入通义千问** — LangChain 生态原生支持、切换模型零成本（仅改 base_url + api_key）
- **④ 单文件 app.py**（2000+ 行）— 共享 session_state 方便，代价是不利于协作
- **⑤ 本地 sentence-transformers**（非 API Embedding）— 免费、离线、中文模型够用

四、性能指标

实测数据：

- LLM 单次响应：**1-3 秒**（取决于提示长度）
- ReAct 完整循环：**5-15 秒**（平均 2-3 轮工具调用）
- PDF 处理：**10 页约 3 秒**（切段+向量化）
- 向量搜索：**< 50ms**（FAISS 内积搜索）
- Embedding 模型加载：**首次 5-10 秒**（@st.cache_resource 后零延迟）

五、Harness Engineering 在哪体现

把裸模型包装成可用 Agent 的"外壳"：

- **工具定义**（react_tools.py）— 6 个工具的 Schema + Executor 双轨
- **Agent 循环**（react_agent.py）— yield 事件流 + 迭代控制
- **对话管理**（base_agent.py）— 消息类型流转 + 上下文维护
- **错误处理** — try/except 包裹 LLM 调用 + 友好错误事件
- **迭代限制** — max_iterations=5 防 API 耗尽

这些"外壳"让 LLM 从"被动回答工具"变成"主动调度的智能体"。